That syntax is straightforward. It says “We want to add a new action named `who_bought`, invoked via an HTTP `GET`. It applies to each member of the collection of products.”

Instead of specifying `:member`, if we instead specified `:collection`, then the route would apply to the collection as a whole. This is often used for scoping; for example, you may have collections of products on clearance or products that have been discontinued.

Nested Resources

Often our resources themselves contain additional collections of resources. For example, we may want to allow folks to review our products. Each review would be a resource, and collections of reviews would be associated with each product resource. Rails provides a convenient and intuitive way of declaring the routes for this type of situation:

```
resources :products do
  resources :reviews
end
```

This defines the top-level set of product routes and additionally creates a set of subroutes for reviews. Because the review resources appear inside the products block, a review resource *must* be qualified by a product resource. This means that the path to a review must always be prefixed by the path to a particular product. To fetch the review with ID 4 for the product with an ID of 99, you’d use a path of `/products/99/reviews/4`.

The named route for `/products/:product_id/reviews/:id` is `product_review`, not simply `review`. This naming simply reflects the nesting of these resources.

As always, you can see the full set of routes generated by our configuration by using the `rails routes` command.

Routing Concerns